

Plan Projet — MED (Metro, Efrei, Dodo)

Optimisation des temps de trajet dans le réseau de transport parisien

Plateforme cible : application mobile native (cross-platform)

1. Choix techniques (justifiés, comme exigé par l'AO)

1.1 Langage & framework : Flutter / Dart

Contrainte de l'AO	Réponse Flutter/Dart
Rapidité	Dart compilé AOT en code natif ARM → performances proches du natif pur
Sobriété en ressources	Calcul d'itinéraire 100 % on-device, hors-ligne : zéro serveur, zéro requête réseau → consommation minimale, argument carbone direct
Facilité de déploiement	Une seule base de code → APK Android + iOS ; l'évaluateur lance la démo avec <code>flutter run</code> ou installe l'APK
Reproductibilité	<code>flutter test</code> natif, CI GitHub Actions standard (test + analyse + build APK)
Fluidité UX	Moteur de rendu 60/120 fps, animations natives — l'AO note la fluidité du parcours

Alternatives écartées (à argumenter en soutenance) :

- *Kotlin natif Android* : excellent, mais exclut iOS et réduit la portée de la démo.
- *React Native (JS)* : pont JS → surcoût CPU/mémoire sur le calcul de graphe, pénalisant sur le critère consommation.
- *Swift iOS* : nécessite un Mac pour builder, frein à la reproductibilité côté évaluateur.

1.2 Pipeline de données : Python (hors application)

Script de préparation exécuté une fois, jamais à l'exécution : GTFS IDFM → nettoyage → construction du graphe pondéré → export en binaire compact embarqué dans l'app (asset).
→ L'app ne parse jamais de CSV au démarrage : chargement rapide, mémoire maîtrisée.

1.3 Données

- **Source** : GTFS Île-de-France Mobilités (data.iledefrance-mobilites.fr), licence **ODbL** — à citer dans le README.
 - **Périmètre V1** : métro (14 lignes + 3bis/7bis) ; extension RER/tram en V2+.
 - **À documenter** (exigence AO) : date de récupération, hypothèses de pondération (temps inter-stations, pénalité de correspondance fixe ~4 min), limites géographiques, valeurs manquantes.
 - Aucune donnée personnelle, aucune clé API dans le dépôt.
-

2. Cœur algorithmique

Modélisation : graphe orienté pondéré. Nœuds = couples (station, ligne) pour modéliser finement les correspondances ; arêtes = trajets inter-stations (poids = temps) + arcs de correspondance (poids = pénalité).

Attendu AO	Algorithme	Complexité	Justification
Tester la connexité	Parcours BFS/DFS, composantes connexes	$O(V+E)$	Vérifie l'intégrité du réseau après import ; détecte stations isolées (données corrompues)
Afficher l'arborescence	Arbre couvrant minimal (Prim, tas binaire)	$O(E \log V)$	Visualisation de la structure du réseau exigée par l'AO
Plus court chemin	Dijkstra avec file de priorité (tas binaire)	$O((V+E) \log V)$	Adapté au volume RATP (~1 600 nœuds éclatés) ; un Dijkstra naïf $O(V^2)$ serait éliminatoire
Optimisation	A* avec heuristique = distance géodésique / vitesse max du métro	$\leq$ Dijkstra	Heuristique admissible (jamais surestimée) → optimalité conservée, espace exploré réduit

Implémentés **à la main** (pas de librairie de graphes), avec **tests unitaires** sur : connexité, optimalité du chemin, admissibilité de l'heuristique, cas limites (départ = arrivée, station inexistante).

Mesures de performance (critère noté) : temps de calcul moyen/p95 sur 1 000 requêtes aléatoires, pics mémoire, estimation énergie (profiling Android Battery Historian / adb batterystats + extrapolation Watts). Benchmark Dijkstra vs A* documenté.

3. Architecture du dépôt

```

med/
├── data-pipeline/    # Python : GTFS → graph.bin (asset)
├── core/             # Package Dart pur : graphe, Dijkstra, A*, BFS, Prim
│   └── test/         # Tests unitaires algo (priorité n°1)
├── app/              # Flutter : UI, navigation, visualisation
├── benchmarks/       # Scripts de mesure perf/conso + résultats
├── .github/workflows/ # CI : analyse + tests + build APK
└── README.md         # Prérequis, installation, données, démo en 3 commandes

```

Séparation stricte **core (algo)** / **app (UI)** : le cœur algorithmique se teste et se benchmarke sans interface — réponse directe au point de vigilance « ne pas se limiter à une interface ».

4. Interface — 4 écrans (maquettés dans Figma)

1. **Accueil / Recherche** — champs Départ/Arrivée avec autocomplétion, inversion en un tap, accès rapide aux stations récentes, fond carte du réseau stylisée.
2. **Résultats** — 3 propositions comparées : *Le plus rapide* / *Moins de correspondances* / *Le plus sobre*, chacune avec durée, badges de lignes colorés (codes couleur RATP), CO₂ évité.
3. **Détail itinéraire** — timeline verticale colorée par ligne, stations intermédiaires repliables, correspondances signalées avec temps de marche, barre de progression.
4. **Impact & Performance** — CO₂ évité cumulé vs voiture, et **transparence algo** : temps de calcul affiché (« itinéraire calculé en 8 ms »), graphe du réseau/arborescence visualisable — vitrine du travail algorithmique en démo.

Principes UX : mobile une main (actions en bas d'écran), thème sombre type RATP nuit, animations de transition fluides, 2 taps maximum entre ouverture et résultat.

5. Planning par versions (profondeur > étendue)

Version	Contenu	Critère de sortie
V1 — socle	Pipeline data + graphe + Dijkstra + connexité + tests unitaires + README + CI verte	Itinéraire correct calculé en CLI/écran minimal, reproductible en 3 commandes
V2 — produit	UI Flutter complète (4 écrans), A* + benchmark vs Dijkstra, arborescence (Prim) visualisée	Démo utilisateur fluide de bout en bout

V3 — optimisation	Mesures énergie (Watts), tuning mémoire (structures compactes), pénalités de correspondance affinées, comparatif CO ₂	Rapport de perf chiffré intégré à la soutenance
------------------------------	--	--

6. Conformité aux points d'attention de l'AO

- ☒ Langage justifié par contraintes (rapidité, sobriété, déploiement) — §1.1
- ☒ Algorithme ≠ dashboard : core algo séparé, testé, benchmarké — §2, §3
- ☒ Reproductibilité : tests, README 3 commandes, CI GitHub Actions — §3, §5
- ☒ Performance & consommation mesurées en Watts — §2, §5 (V3)
- ☒ Données documentées (licence ODbL, hypothèses, limites) — §1.3
- ☒ Pas de données personnelles ni de secrets dans le dépôt — §1.3